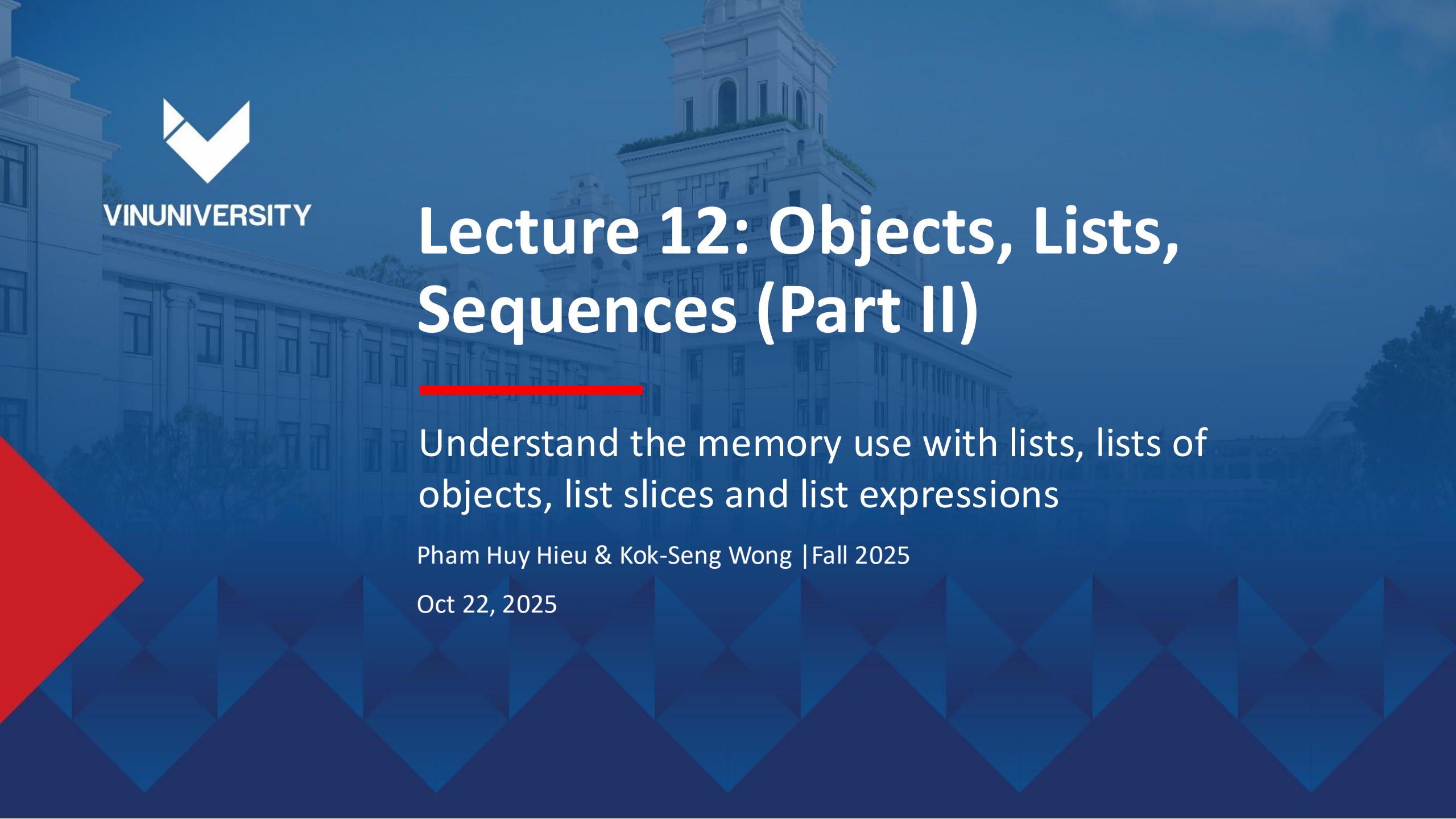


Lecture 12: Objects, Lists, Sequences (Part II)

Understand the memory use with lists, lists of objects, list slices and list expressions

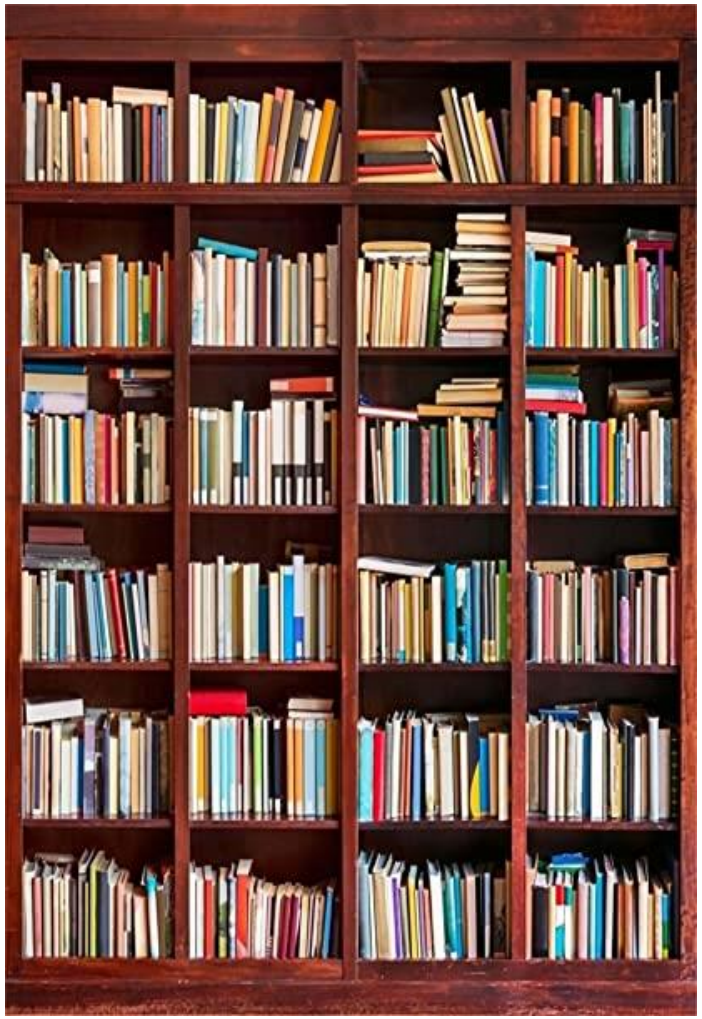
Pham Huy Hieu & Kok-Seng Wong |Fall 2025

Oct 22, 2025

LISTS - REVIEW

- How to define a list and explain the differences between strings and lists.
- Use indexing and slicing to access list elements in Python.
- Manipulate lists with the built-in methods.
- Understand key operations in lists.

MOTIVATIONAL EXAMPLE



[source]



[source]

LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- understand memory use with list in Python.
- understand and apply nested list method.
- use enumerate to keep track of index and element values.
- make a list comprehension in Python.

CLASS PRACTICE (REVIEW)

Sample Code:

```
>>> p = [2, 4, 6]
>>> p += [10]
>>> p
>>> q = [1, 3, 5]
>>> q += 9
>>> q
```

Output:

CLASS PRACTICE (REVIEW)

Sample Code:

```
>>> p = [2, 4, 6]
>>> p += [10]
>>> p
>>> q = [1, 3, 5]
>>> q += 9
>>> q
```

Output:

```
[2, 4, 6, 10]
```

CLASS PRACTICE (REVIEW)

Sample Code:

```
>>> p = [2, 4, 6]
>>> p += [10]
>>> p
>>> q = [1, 3, 5]
>>> q += 9
>>> q
```

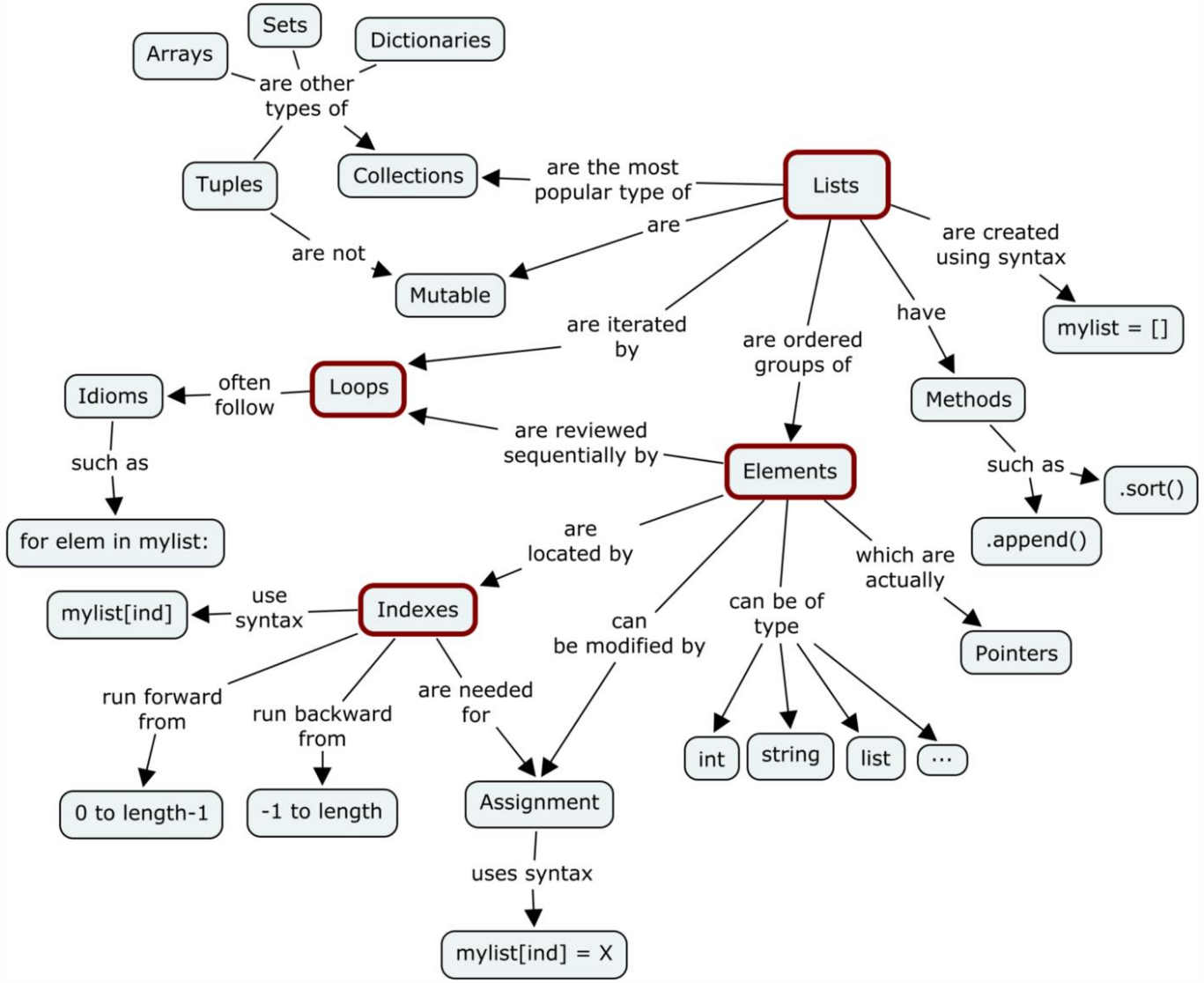
Output:

```
[2, 4, 6, 10]
TypeError: 'int' object
is not iterable
```

CLASS PRACTICE (REVIEW)

```
>>> list1 = list(range(10))
>>> list1
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list2 = list(range(3,15))
>>> list2
[3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14
```

PYTHON LISTS (REVIEW)

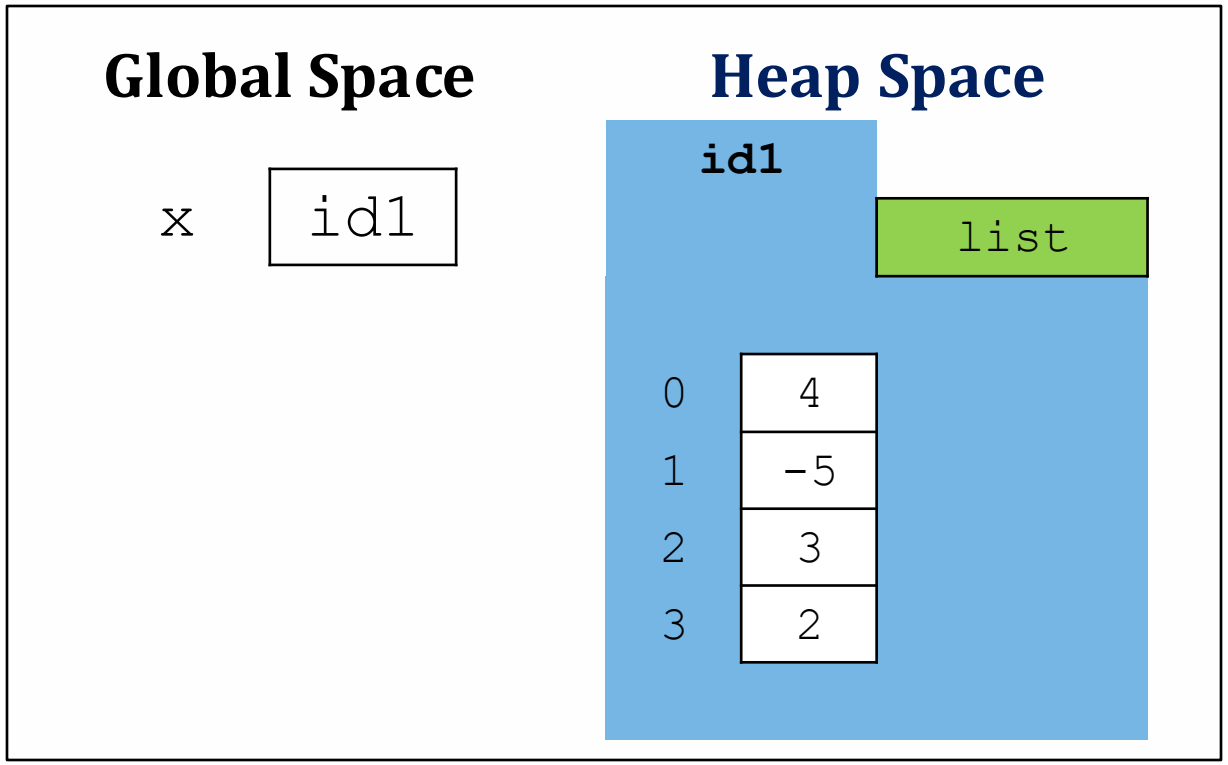


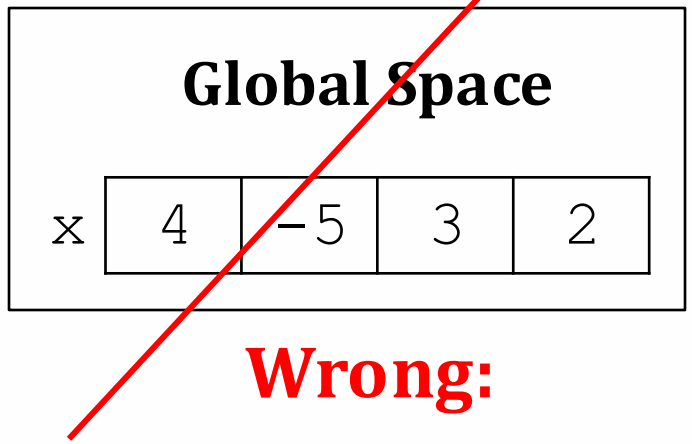
- The list is a single object that manages an internal array of references to other objects.
- A list itself is one object in memory.
- It has its own memory address, identity in Python.
- It doesn't directly hold all the items inside its structure, instead, it keeps references (pointers) to them.

LISTS REPRESENTATION

```
x = [4, -5, 3, 2]
```

Correct:





Wrong:

OBJECTS

- Python supports different kinds of data:

object:	`"abc"`	`3.14159`	`[1, 3, 5]`	`223`
type:	`string`	`float`	`list`	`int`

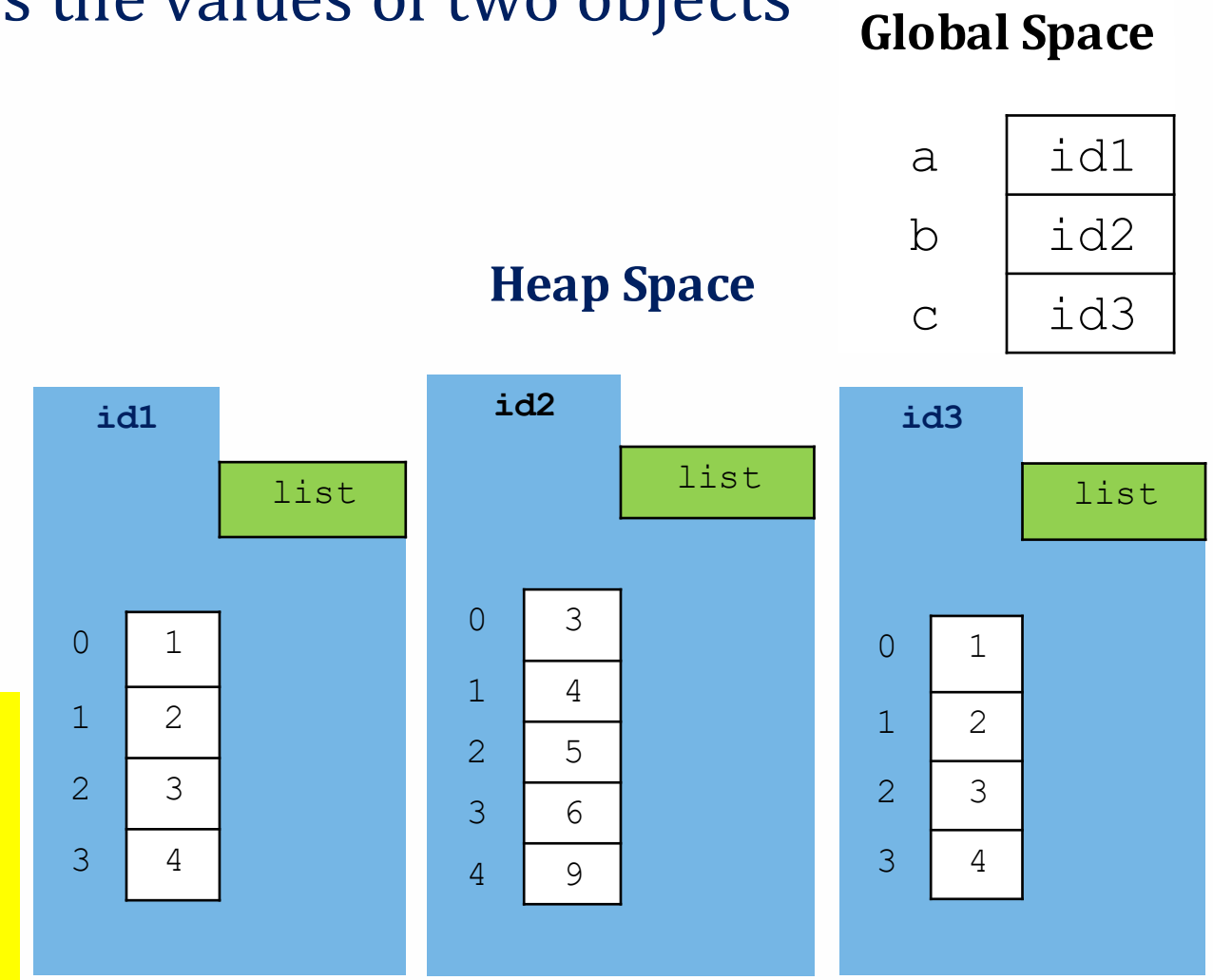
- Each object has the following properties:
  1. A type
  2. An internal data representation
  3. A set of procedures for interaction with the object
- An object is an **instance** of a type

COMPARING TWO LISTS

- Equality operator (==) compares the values of two objects

```
>>> a = [1, 2, 3, 4]
>>> b = [3, 4, 5, 6, 9]
>>> c = [1, 2, 3, 4]
>>> a == c
True
>>> a == b
False
```

- equality operator (**==**) compares the values of both the operands and checks for value equality.
- '**is**' operator checks whether both the operands refer to the same object or not (check if they are in the same memory location)



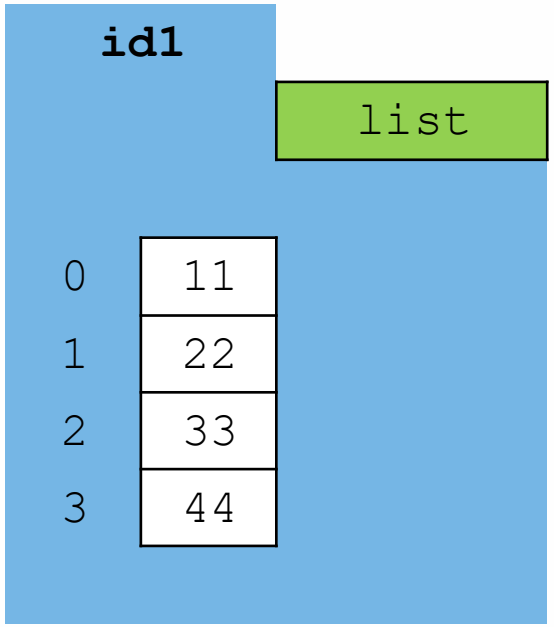
ALIASING LISTS

```
>>> list_1 = [11, 22, 33, 44]
>>> list_2 = list_1
>>> list_2
[11, 22, 33, 44]
```

Global Space

list_1	id1
list_2	id1

Heap Space



- List aliasing occurs when multiple variables reference the same list.
- `list_1` and `list_2` are variables that refer to the same list object.

THE IS OPERATOR

- `is` operator compares the identity (memory address) of two objects - not their contents.

```
>>> list_1 = [11, 22, 33, 44]
>>> list_2 = list_1
>>> list_1 is list_2
True
>>> list_2
[11, 22, 33, 44]
>>> list_2 = [55, 66]
>>> list_3 = [55, 66]
>>> list_2 is list_3
False
```

means both names refer to the same list object.

list_1 and list_2 refer to the same object

Here, you create two separate list objects that happen to have the same elements.

list_1 and list_3 have the same elements BUT refer to different objects

COPYING A LIST

- **`list()`**: makes a copy of a list

```
>>> alphabets = ["t", "w", "h", "k"]
>>> alphabets_copy = list(alphabets)
>>> alphabets_copy
['t', 'w', 'h', 'k']
>>> alphabets_copy is alphabets
False
```

EXTEND A LIST

- **`extend()`**: adds a sequence of elements to the end of a list

```
>>> SEA_country = ["Thailand", "Vietnam", "Malaysia"]
>>> SWA_country = ["Yemen", "Saudi Arabia", "Kuwait", "Qatar"]
>>> SEA_country.extend(SWA_country)
>>> SEA_country
['Thailand', 'Vietnam', 'Malaysia', 'Yemen', 'Saudi Arabia',
'Kuwait', 'Qatar']
```

Same as: `SEA_country + SWA_country`

CLASS PRACTICE

Sample Code:

```
>>> language = ["Java", "Python", "C++"]
>>> new_language = ["Pascal", "VB"]
>>> ?
```

Output:

```
>>> language
['Java', 'Python', 'Pascal', 'VB', 'C++']
```

CLASS PRACTICE

Sample Code:

```
>>> a = ["t", "w", "h", "k"]
>>> b = a[1:3]
>>> print(b)
>>> c = b.sort()
>>> print(b)
>>> print(c)
```

[demo]

CLASS PRACTICE

Sample Code:

```
>>> a = ["t", "w", "h", "k"]
>>> b = a[1:3]
>>> print(b)
>>> c = b.sort()
>>> print(b)
>>> print(c)
```

Final output:

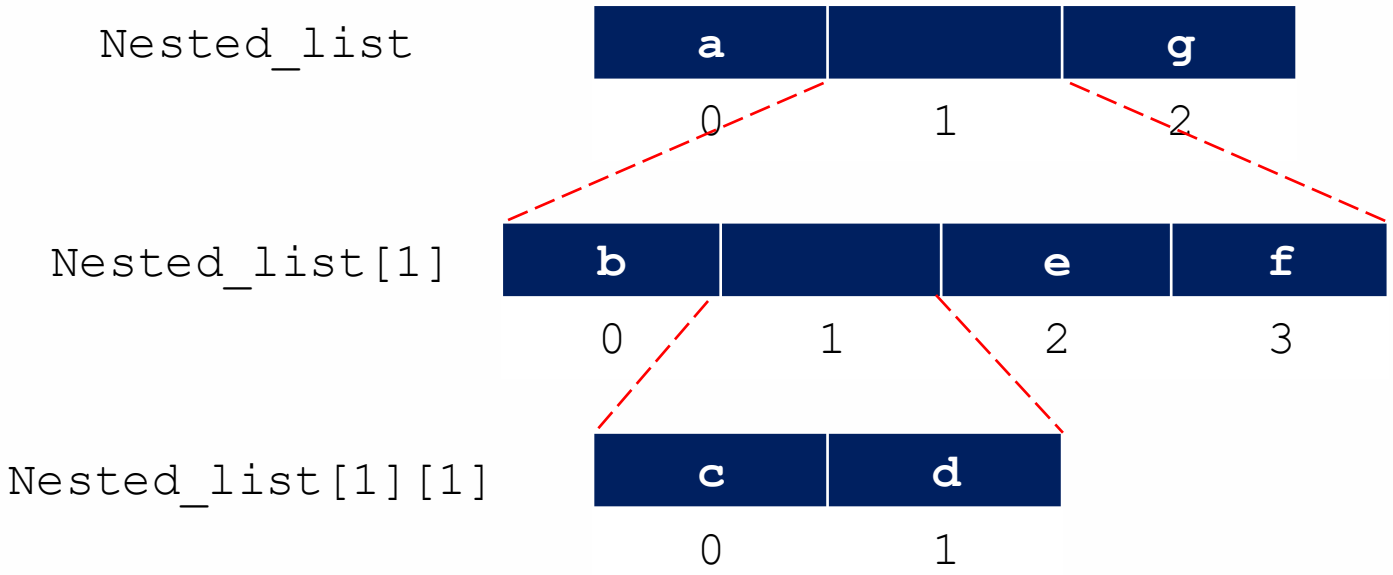
```
['w', 'h']
['h', 'w']
None
```

[demo]

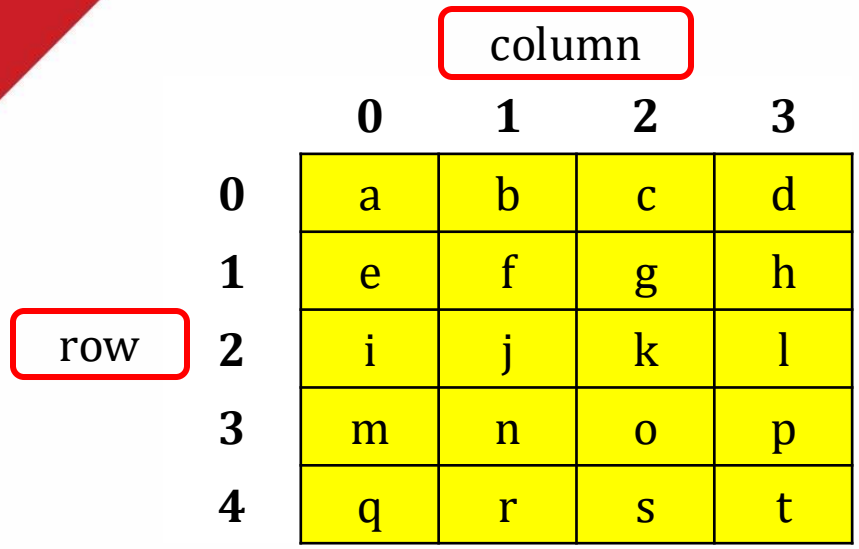
NESTED LIST

A nested list: a list contains another list (sub-list)

```
nested_list = ["a", ["b", ["c", "d"], "e", "f"], "g"]
```



2-DIMENSIONAL LISTS




```
my_2D_list = [
      ["a", "b", "c", "d"],
      ["e", "f", "g", "h"],
      ["i", "j", "k", "l"],
      ["m", "n", "o", "p"],
      ["q", "r", "s", "t"]
]
```

```
print(my_2D_list[3][3])
print(my_2D_list[0][3])
print(my_2D_list[1][2])
```

CLASS PRACTICE

Sample Code:

```
X = [1, 2, [33, 44,
     [555, 666]], 7, 8]


print(X[3])
print(X[2])
print(X[2][2])
print(X[2][2][0])
Print(X[-2])
print(X[-3][-1][-2])
```

Output:

2-DIMENSIONAL LISTS: QUIZ

Write a Python program that prints out all elements of a 3×3 matrix, one by one. Note that the matrix elements are generated randomly.

Input:

```
matrix = [
    [4, 7, 1],
    [3, 9, 5],
    [8, 2, 6]
]
```

Expected Output:

4 7 1 3 9 5 8 2 6

Submit your solution to Canvas, under Quiz Lecture 12

ENUMERATE()

- Use `enumerate()` to keep track of index and element values.
- `enumerate()` adds a counter to an iterable and returns it as an enumerate object, which yields pairs of: `(index, element)`
- The `enumerate()` takes two parameters:
  1. **iterable** - a sequence like lists, tuples, or strings.
  2. **start** (`begin`) - starts counting from `begin` number. By default, `enumerate()` starts with 0 if start parameter is omitted.

```
enumerate(iterable, start=0)
```

ENUMERATE()

- `enumerate()` returns a tuple with two values for each iteration:
  - The index (position) of the element.
  - The element itself.
- Index starts at 0 by default, but we can specify a different starting index if needed.

ENUMERATE() EXAMPLE

```
x = ["t", "w", "h", "k"]
for i in enumerate(x):
    print(i)
```

By default start is set to 0

```
(0, 't')
(1, 'w')
(2, 'h')
(3, 'k')
```

```
x = ["t", "w", "h", "k"]
for i in enumerate(x, 5):
    print(i)
```

start is set to 5

```
(5, 't')
(6, 'w')
(7, 'h')
(8, 'k')
```

```
x = ["t", "w", "h", "k"]
for i, item in enumerate(x):
    print(item)
```

```
t
w
h
k
```

```
x = ["t", "w", "h", "k"]
for i, item in enumerate(x, 5):
    print(i, item)
```

```
5 t
6 w
7 h
8 k
```

LIST COMPREHENSION

- A concise way to create lists in Python.
- Can define a list based on an existing iterable (list, tuple, or string).
- Can perform operations on each element while filtering them based on a condition.

```
[expression for item in iterable if condition]
```

The operation to be performed on each element.

The variable representing each element in the iterable.

The original sequence

A filtering condition that determines whether the element should be include in the new list.

```
squares = [x**2 for x in range(1, 6)]
```

LIST COMPREHENSION EXAMPLE 1

Iterating through a string with for-loop

```
country = "Vietnam"
letters_country = []

for letter in country:
    letters_country.append(letter)

print(letters_country)
```

```
['V', 'i', 'e', 't', 'n', 'a', 'm']
```

Iterating through a string with list comprehension method

```
country = "Vietnam"
letters_country = [letter for letter in country]
print( letters_country)
```

```
['V', 'i', 'e', 't', 'n', 'a', 'm']
```

LIST COMPREHENSION EXAMPLE 2



```
squares = [x**2 for x in range(1, 6)]
```

```
[1, 4, 9, 16, 25]
```

```
even_squares = [x**2 for x in range(1, 6) if x % 2 == 0]
```

```
[4, 16]
```

```
sentence = "List comprehensions are powerful"
first_letters = [word[0] for word in sentence.split()]
```

```
['L', 'c', 'a', 'p']
```

```
nested_list = [[1, 2], [3, 4], [5, 6]]
flattened_list = [item for sublist in nested_list for item in sublist]
```

```
[1, 2, 3, 4, 5, 6]
```

THANK YOU!